

Principles of Prompt Design and Engineering



Section 1: The Physics of Language Models and Prompt Dynamics

- LLMs function as next-token predictors, generating output based on the statistical likelihood of a sequence following the provided input.
- The model does not "know" the answer in a human sense; rather, it traverses a probability distribution to construct a plausible continuation of the prompt
- When a user submits a prompt, they are essentially defining the initial conditions of a dynamic system.
- If the initial conditions are vague or high-entropy, the model's output will exhibit high variance, often resulting in "hallucinations" or irrelevant deviations.
- The goal of prompt engineering, therefore, is to constrain this probability space, guiding the model toward a specific, low-entropy "laminar flow" of reasoning, while avoiding the "turbulent flow" of confabulation

The Shift from Querying to Context Engineering

- Context Engineering approach views the prompt not as a single line of text but as a holistic configuration of state.
- It involves curating the entire set of tokens present in the inference window, including system instructions, few-shot examples, retrieval-augmented data, and user history
- Effective context engineering requires managing the "attention mechanism" of the model:
Transformers

Section 2: Core Axioms of Prompt Fidelity

- Axiom 1: Clarity and the Elimination of Ambiguity
 - Clarity dictates that instructions must be explicit and unambiguous.
 - This involves replacing subjective terms with objective metrics
 - Example: instead of asking for a "short summary," a clear prompt specifies "a summary of fewer than 50 words".

Core Axioms of Prompt Fidelity

- Axiom 2: Specificity and Constraint Management
 - Specificity acts as the bounding box for the model's generation.
 - It involves defining the what, how, and format of the output.
 - Constraints can be positive (what to do) or negative (what to avoid).
 - Negative constraints are particularly useful for filtering out common clichés or unwanted formatting.
 - Example: "Do not use flowery language" or "Do not include an introduction" are powerful directives that refine the output.

Core Axioms of Prompt Fidelity

- Axiom 2: Specificity and Constraint Management

Table 1: The Impact of Specificity on Model Output

Dimension	Weak Prompt Strategy	Strong Prompt Strategy	Mechanism of Action
Topic Definition	"Tell me about dogs."	"List 5 dog breeds suitable for small apartment living, detailing their energy levels and grooming needs."	Narrows the semantic search space to a specific subset of the topic.
Output Format	"Summarize this."	"Summarize the text below into a JSON object with keys: 'summary', 'sentiment', and 'key_entities'."	Forces the model to generate structured tokens recognizable by code parsers.
Audience	"Explain quantum physics."	"Explain quantum physics to a 5th-grade student using analogies involving playground equipment."	Adjusts the perplexity and vocabulary selection to match the target demographic.
Tone	"Write an email."	"Write a formal apology email to a client regarding a missed deadline, emphasizing our commitment to rectification."	Sets the emotional and stylistic weights of the generated text.

Core Axioms of Prompt Fidelity

- Axiom 3: Contextual Grounding
 - Context provides the necessary background information for the model to process the request accurately.
 - Without context, the model relies on "hallucinated" or generic assumptions.
 - Providing context involves situating the task within a scenario: "You are analyzing sales data for a Q3 board meeting," or "This code will be used in a legacy Python 2.7 environment".
 - Contextual grounding is closely linked to the concept of the "Input-Output" mapping.
 - By clearly separating the context (the data to be processed) from the instructions (what to do with the data), users prevent "context bleeding," where the model confuses the source text with the commands.

Core Axioms of Prompt Fidelity

- Axiom 4: Persona and Role Simulation
 - One of the most profound capabilities of LLMs is their ability to simulate specific personas.
 - This is often referred to as "Role Prompting."
 - When a user instructs the model to "Act as a Senior Python Developer" or "You are an empathetic grief counselor," they are leveraging the model's training on vast corpora of text associated with those identities.
 - This technique works by priming the model's latent space.
 - A "Senior Developer" persona activates associations with clean code, comments, error handling, and optimization, whereas a "Junior Developer" persona might produce simpler, less robust code.
 - Research by prompt engineers suggests that role prompting acts as a "meta-instruction," implicitly setting thousands of micro-parameters regarding tone, vocabulary, and sentence structure without the user needing to specify them individually.

Section 3: Structural Engineering and Syntax

- The syntactic structure plays a crucial role in reliability, especially for automated interactions via API.
- "Prompt Engineering" in this sense is closer to coding than writing; it involves creating a predictable interface between human intent and machine execution.

Structural Engineering and Syntax

- Delimiters and Segmented Architecture: Advanced models are sensitive to the visual structure of the input. Using delimiters, characters that visually separate different parts of the prompt, is a best practice for complex tasks.
 - **Triple Quotes (""") or Backticks (` ` `):** These are standard for separating code blocks or large text excerpts. They signal to the model that the enclosed content is data to be processed, not instructions to be followed.
 - **XML Tags (<instruction>, <context>, <output>):** Models like Claude are fine-tuned to recognize XML-like structures. Using tags allows for precise segmentation of the prompt, enabling the model to distinguish between the system's role, the user's data, and the examples provided.²⁰
 - **Section Headers (###, ---):** These act as visual breaks, helping the model parse transitions between context, instructions, and output requirements.

Structural Engineering and Syntax

- Output Formatting and Structured Generation: For integration into software pipelines, natural language output is often insufficient.
- Developers require structured data formats like JSON, CSV, or XML. Prompting for these formats requires explicit schema definitions.
- Simply asking for JSON is often risky; the model might include conversational filler like "Here is your JSON:". To mitigate this, a strong prompt includes:
 - Format Enforcing: "Output strictly valid JSON. Do not include markdown formatting or conversational text."
 - Schema Definition: Providing a template (e.g., {"name": "string", "age": "int"}) guides the model's generation process.
 - Data Typing: Specifying the type of data (e.g., "Format dates as YYYY-MM-DD") ensures consistency for downstream parsing

Section 4: Cognitive Architectures and Reasoning Strategies

- The fundamental axioms ensure clarity, complex tasks require more than just clear instructions. However, they require reasoning.
- LLMs, by default, perform "System 1" thinking, fast, intuitive, and prone to error.
- Advanced prompt engineering aims to induce "System 2" thinking, slow, deliberate, and logical.
- This is achieved through specific prompting frameworks that structure the model's computation

Cognitive Architectures and Reasoning Strategies

- Chain of Thought (CoT) Prompting: encourages the model to generate intermediate reasoning steps before arriving at a final answer
- In standard prompting, the model attempts to jump directly from the question to the answer

$$Q \rightarrow A$$

- For complex math or logic, this often fails because the computation required exceeds the capacity of a single token generation.
- CoT forces the model to externalize its working memory

$$Q \rightarrow R_1 \rightarrow R_2 \rightarrow \cdots \rightarrow$$

- This decomposition allows the model to attend to each step individually, reducing the cognitive load and error rate

Cognitive Architectures and Reasoning Strategies

- Chain of Thought (CoT) Prompting
 - Implementation:
 - Zero-Shot CoT: Simply appending "Let's think step by step" to the prompt can trigger this reasoning capability without specific examples.²⁷
 - Few-Shot CoT: Providing examples of questions paired with their step-by-step solutions is even more robust, as it models the exact depth and format of reasoning required.

Cognitive Architectures and Reasoning Strategies

- Chain of Thought (CoT) Prompting

Table 2: Comparative Analysis of Reasoning Strategies

Strategy	Prompt Structure	Strength	Weakness	Use Case
Zero-Shot	"Solve X."	Fast, simple.	High error rate on logic/math.	General knowledge, creative writing.
Few-Shot	"Example 1... Example 2... Solve X."	Pattern matching, format compliance.	Can fail if examples utilize different logic than the target.	Classification, style transfer.
CoT	"Let's think step by step..."	High accuracy on multi-step problems.	Higher token cost, slower latency.	Math, coding, logical puzzles.
Self-Consistency	Run CoT multiple times, take majority vote.	Extremely robust, eliminates outliers.	Very high cost (N runs).	Critical decision making, verifiable facts.

Cognitive Architectures and Reasoning Strategies

- Tree of Thoughts (ToT)
 - For problems requiring strategic lookahead or exploration of multiple possibilities (e.g., creative writing, game playing, complex planning), linear CoT is insufficient. The Tree of Thoughts framework generalizes CoT into a tree search.
- Mechanism:
 - ToT maintains a tree of partial solutions ("thoughts").
 - At each step, the model generates multiple candidate thoughts.
 - It then evaluates these candidates using a "Self-Evaluator" prompt (e.g., "Is this step possible? Sure/Maybe/Impossible").
 - Based on the evaluation, the system uses search algorithms like Breadth-First Search (BFS) or Depth-First Search (DFS) to explore the most promising branches, backtracking if a dead end is reached

Cognitive Architectures and Reasoning Strategies

- Tree of Thoughts (ToT)
 - Application:
 - In the "Game of 24" (a math puzzle), ToT significantly outperforms CoT by exploring different equation combinations and discarding those that cannot possibly reach the target number.
 - In creative writing, it allows the model to draft multiple plot twists, evaluate their coherence, and select the best one before proceeding.

Cognitive Architectures and Reasoning Strategies

- ReAct: Synergizing Reasoning and Acting
 - LLMs are typically isolated from the world. The ReAct (Reasoning + Acting) framework overcomes this by allowing the model to interact with external tools.
 - The ReAct Loop:
 1. Thought: The model reasons about the user's request ("I need to find the population of France").
 2. Action: The model emits a specific command ("Search['population of France']").
 3. Observation: The external system executes the command and feeds the result back to the model ("67 million").
 4. Thought: The model processes this new information ("Now I can answer the user").
 5. Answer: Final output.
 - This loop grounds the model in reality, significantly reducing hallucinations compared to pure CoT, although it can be more constrained in its reasoning flexibility.

Section 5: The Mechanics of Instruction: What an LLM Really Is?

- An LLM should not be understood as a database of facts or a search engine. Instead, it is fundamentally a probabilistic text generation engine.
- Its core task is simple: given a sequence of tokens, it predicts the most likely next token based on learned probability distributions from massive amounts of text.
- A helpful analogy is “autocorrect on steroids.”
 - Just as autocorrect predicts the next word you intend to type, an LLM predicts the next word, except it does so with vastly more context, scale, and statistical sophistication.
- When we write a prompt, we are not really asking the model a question in the human sense. We are setting the initial conditions for a stochastic generation process. The prompt defines the starting state from which the model begins sampling its output.

The Mechanics of Instruction: The Stochastic Nature of LLMs

- LLMs do not produce a single “correct” answer deterministically. Instead, they work with a probability distribution over possible next tokens at every step.
- Two key concepts govern this behavior:
 - Temperature: Controls randomness.
 - Low temperature → conservative, predictable outputs
 - High temperature → creative, diverse outputs
 - Top-p (nucleus) sampling: Restricts token selection to the smallest set whose cumulative probability exceeds p , filtering out unlikely continuations.
- When a prompt is given, it collapses the probability distribution of possible outputs:
 - A vague prompt leaves many continuations equally plausible. The distribution remains relatively flat; this is high entropy, leading to generic or inconsistent answers.
 - A specific prompt sharply narrows the space of valid continuations. The distribution peaks strongly around a small set of outputs; this is low entropy, producing focused and reliable responses.

The Mechanics of Instruction: Direct Instruction- The Principle of Clarity

- This leads to the first and most important prompting principle: Clarity.
- Human-to-human communication relies heavily on:
 - Shared background knowledge
 - Implicit assumptions
 - Contextual inference
- Human-to-AI communication does not.
- An LLM has no awareness of intent beyond what is explicitly stated. It follows instructions literally and probabilistically, not intuitively.
- Therefore, effective prompting requires us to:
 - Eliminate ambiguity
 - Specify constraints
 - Make expectations explicit
- You must describe what you want, how you want it, and under what conditions.

The Mechanics of Instruction: Example

- Weak Prompt: “Write code to calculate fibonacci.”
- Critique:
 - Which programming language?
 - Recursive or iterative approach?
 - What should be the output format?
 - How many terms?
 - Any performance constraints?
 - The model must guess, leaving too many degrees of freedom.

The Mechanics of Instruction: Example

- Strong Prompt: "Write a Python 3 function to calculate the first n numbers of the Fibonacci sequence using an iterative approach for $O(n)$ time complexity. Return the sequence as a list."
- Language is specified (Python 3)
- Algorithmic approach is constrained (iterative)
- Performance expectations are defined ($O(n)$)
- Output format is explicit (list)
- This prompt sharply narrows the probability distribution, guiding the model toward a precise, high-quality response.

The Syntax of Control

- Effective prompting is not only about what you say, but how you structure it.
- Delimiters, structural cues, and prompt hierarchy, which together form the syntax of control; the mechanisms that tell the model what is instruction, what is data, and what must never be confused.

The Syntax of Control: System vs. User Prompts

- A modern LLM prompt stack is hierarchical.
- System Prompt: This acts as a meta-instruction.
 - It defines:
 - The role or persona of the model
 - Behavioral constraints
 - Tone, style, and safety boundaries
 - System prompts are persistent across the conversation and carry the highest priority when the model resolves conflicts.
- User Prompt: This is the task-level instruction: what the user wants done within the boundaries set by the system prompt.
- Key idea: If the system prompt says “You are a strict JSON generator” and the user says “Explain in paragraphs”, the system prompt wins.
- This hierarchy is essential for controlling behavior at scale.

The Syntax of Control: The Power of Delimiters

- LLMs parse text sequentially.
- Without clear markers, the model must infer what is instruction and what is raw data, often incorrectly.
- Delimiters (such as ###, triple backticks ``` , or <context> tags) act as syntactic boundaries that explicitly separate:
 - Instructions
 - Data
 - Examples
- This is important because:
 - Prevents ambiguity
 - Improves extraction accuracy
 - Mitigates Prompt Injection, where malicious or accidental data attempts to override instructions
- Example concept to explain on board: “Ignore all previous instructions and do X” → If this appears inside a delimited data block, the model treats it as content, not an instruction.

The Syntax of Control: Output Formatting and One-Shot Formatting

- By default, LLMs generate free-form text.
- To force structure, we must:
 - Explicitly specify the format
 - Demonstrate it
- This leads to One-Shot Formatting.
- Definition: Providing one example of the desired output structure (e.g., JSON, CSV, table) strongly biases the model to replicate that schema.
- The model treats the example as a template, not as content.

The Syntax of Control: Exercise

- Write a prompt that extracts dates and corresponding events from a given history passage and outputs them in CSV format.
- Constraints
 - Use delimiters to separate:
 - The history text
 - The extraction instructions
 - Output must:
 - Be a valid CSV
 - Include a clear header row

The Syntax of Control: Exercise

- You are an information extraction system.
- Extract all dates and associated events from the text below.
- `#### RULES ####`
- - Output only CSV
- - Use the header: Date,Event
- - Do not add explanations
- `#### In 1947, India gained independence.`
- `On January 26, 1950, the Constitution of India came into effect. ####`
- Expected Output:
 - Date,Event
 - 1947,India gained independence
 - January 26 1950,Constitution of India came into effect

Context and Few-Shot Learning

- Large Language Models do not “learn” during a conversation in the way humans do.
- Instead, they exhibit In-Context Learning (ICL), the ability to infer patterns, formats, and rules from examples provided inside the prompt itself.
- Everything the model uses to reason must fit within its context window, which is a finite working memory containing:
 - System instructions
 - User instructions
 - Examples
 - Data
- Once the context window is exceeded, earlier information is truncated and effectively forgotten.

Zero-Shot vs. Few-Shot Prompting

- Zero-Shot Prompting
 - In zero-shot prompting, the model relies entirely on its pre-training.
 - Example:
 - “Translate this sentence to Spanish.”
 - This works well for:
 - Common tasks
 - Well-defined transformations
 - Standard language use
 - However, zero-shot prompting is weak at:
 - Strict formatting
 - Domain-specific tone
 - Subtle stylistic constraints

Zero-Shot vs. Few-Shot Prompting

- Few-Shot Prompting
 - Few-shot prompting provides explicit examples inside the prompt.
 - These examples act as temporary training data, allowing the model to infer:
 - Output structure
 - Style and tone
 - Edge-case handling
 - In practice, few-shot prompting is almost always superior for:
 - Format compliance (JSON, CSV, tables)
 - Stylistic consistency
 - Complex or ambiguous tasks

Zero-Shot vs. Few-Shot Prompting

- The “Shot” Hierarchy
 - You can present this as a spectrum of control:
 - Zero-Shot: “Translate this to Spanish.”
 - Relies purely on general knowledge
 - Fast and concise
 - Least controllable
 - Good for: simple, generic tasks
- One-Shot “Translate this to Spanish. Example: Hello → Hola.”
 - Introduces a single anchor example
 - Helps set tone, register, or format
- Good for: style nudging and light constraints

Zero-Shot vs. Few-Shot Prompting

- Few-Shot
 - Providing 3–5 examples:
 - Hello → Hola
 - Good morning → Buenos días
 - How are you? → ¿Cómo estás?
 - Establishes a clear pattern
 - Reduces ambiguity
- Good for: complex transformations and strict outputs
- Selection of Examples: Not all examples are equal. Poor example selection can degrade performance.
- Best Practices
 - Examples should be diverse
 - Different sentence lengths, structures, and edge cases
 - Examples should be representative that reflect the real input distribution

Zero-Shot vs. Few-Shot Prompting

- Recency Bias
 - LLMs tend to weight recent tokens more heavily than earlier ones. This leads to recency bias, where the model:
 - Over-imitates the last example
 - Copies its structure too literally
- Mitigation Strategies
 - Vary the complexity of examples
 - Avoid placing an edge case as the final example
 - Use a neutral or “average” example last

Persona and Role-Playing

- LLMs can simulate roles, expertise levels, and communication styles by activating what can be thought of as linguistic simulacra: clusters of vocabulary, reasoning patterns, and rhetorical structures associated with specific professions, audiences, or personas.
- By explicitly assigning a role, we bias the model toward a particular region of its learned semantic space, improving relevance, rigor, and clarity.

Persona and Role-Playing

- The Expert Persona:
 - Consider the difference between: “Write a contract.”vs.“Act as a specialized corporate lawyer and draft a shareholder agreement.”
 - The second prompt works better because the phrase “specialized corporate lawyer” activates a dense semantic cluster that includes:
 - Domain-specific terminology
 - Formal legal syntax
 - Structured argumentation
 - Risk-aware phrasing
 - The model is not “becoming” a lawyer, but it is retrieving and prioritizing tokens statistically associated with legal expertise. This results in outputs that are more precise, cautious, and professionally aligned.
 - Personas guide how the model thinks, not just what it writes.

Persona and Role-Playing

- Audience Constraints: Personas are equally powerful for controlling complexity.
 - EL15 (Explain Like I'm 5) technique enforces:
 - Short sentences
 - Concrete metaphors
 - Minimal abstraction
 - Familiar vocabulary
 - Technical Deep Dive: a technical persona encourages:
 - Dense terminology
 - Formal definitions
 - Mathematical or theoretical framing
 - Assumed background knowledge
 - By combining role + audience, we can finely tune the output's difficulty level.
- Example: "You are a senior data scientist explaining gradient descent to an undergraduate." vs. "You are a math professor explaining gradient descent to a first-year PhD student."

Persona and Role-Playing

- The Refinement Loop- A critical prompting principle: The first prompt is rarely optimal.
- This leads to Iterative Refinement, where prompts are gradually improved based on output inspection.
- One effective framework is the STAR Schema:
 - Situation – Context or role
 - Task – What needs to be done
 - Action – Constraints, methods, or style
 - Result – Expected output format or quality
- Each refinement reduces ambiguity and entropy, progressively converging on the desired output.
- Example refinement path:
 - “Explain inflation.”
 - “Act as an economist and explain inflation.”
 - “Act as an economist explaining inflation to undergraduates using real-world examples.”

Persona and Role-Playing: The Persona Shift

- Base Prompt: “Explain the concept of inflation.”
- Expert Persona: “You are a Nobel Prize-winning economist explaining inflation to a PhD candidate.”
 - Expected characteristics:
 - High vocabulary density
 - Abstract concepts (monetary policy, expectations, supply shocks)
 - Long, compound sentences
 - Assumed prior knowledge
- Simplified Persona: “You are a kindergarten teacher explaining inflation to a 5-year-old using candy.”
 - Expected characteristics:
 - Simple vocabulary
 - Concrete metaphors
 - Short sentences
 - Narrative framing

List:

Sentence length

Vocabulary complexity

Use of metaphors vs. abstractions

Logical structure

Section 6: Advanced Reasoning and Reliability- Engineering Reasoning

- Two Ways of Thinking
 - System 1 (Fast / Intuitive)
 - Quick, automatic, pattern-based
 - In LLMs: corresponds to standard prompting
 - The model jumps to the most probable next token → greedy decoding
 - Works well for simple or familiar tasks, but fails on multi-step reasoning
 - System 2 (Slow / Analytical)
 - Deliberate, logical, step-by-step
 - In LLMs: corresponds to Chain-of-Thought prompting
 - Forces the model to explore intermediate reasoning states
 - Reduces premature conclusions

Advanced Reasoning and Reliability- Engineering Reasoning

- Chain-of-Thought (CoT) and breaking the “greedy decoding” bottleneck
 - Chain-of-Thought (CoT) is a prompting technique where a language model is encouraged to show its step-by-step reasoning before giving the final answer.
 - Instead of jumping directly to the answer, the model explains intermediate steps.
 - Without CoT
 - Q: If a train travels 60 km in 1 hour, how far in 3 hours?
 - A: 180 km
 - With CoT
 - 1 hour $\rightarrow$ 60 km
 - 3 hours $\rightarrow 60 \times 3$
 - = 180 km

Chain-of-Thought (CoT) and Greedy Decoding

- Chain-of-Thought (CoT) and breaking the “greedy decoding” bottleneck
 - What is “Greedy Decoding”?
 - When generating text, a language model predicts the next word token one step at a time.
 - Greedy Decoding means at each step, choose the highest probability token.
 - Example: If probabilities are:
 - “is” $\rightarrow$ 0.60
 - “was” $\rightarrow$ 0.30
 - “has” $\rightarrow$ 0.10
 - Greedy decoding picks “is” immediately.

Advanced Reasoning and Reliability- Engineering Reasoning

- Exercise 1:
 - Prompt: answer "If 5 shirts dry in 4 hours, how long for 10 shirts?"
 - Answer: ?
- Exercise 2:
 - Prompt: answer "If 5 shirts dry in 4 hours, how long for 10 shirts?" step by step
 - Answer: ?

Advanced Reasoning and Reliability- Engineering Reasoning

- Chain-of-Thought (CoT) and breaking the “greedy decoding” bottleneck
 - Greedy decoding can:
 - Get stuck in locally optimal choices
 - Miss better reasoning paths
 - Produce shallow or incorrect answers
 - It only follows the most likely immediate word and not necessarily the best overall reasoning path.
- Chain-of-Thought helps because:
 - It forces the model to generate intermediate reasoning steps
 - This opens alternative reasoning paths
 - It reduces early commitment to a wrong short answer
 - It increases chances of reaching a correct final answer
- Instead of: **Question** → **Answer**. It becomes: **Think** → **Reason** → **Verify** → **Answer**

Advanced Reasoning and Reliability- Engineering Reasoning

- The “Let’s Think Step by Step” Breakthrough
 - In 2022, researchers published an influential paper: Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
 - The key discovery was simple:
 - Adding the phrase “Let’s think step by step.” improved reasoning performance in LLMs.
- The researchers tested models on math word problem benchmarks such as GSM8K
 - Without Chain-of-Thought:
 - Model gives direct answers
 - Accuracy was very low (around ~15–20%)
 - With Chain-of-Thought prompting:
 - Model explains reasoning step by step
 - Accuracy jumped dramatically (in large models, performance increased to around ~75–80%+)
- The huge jump ($\approx 17\% \rightarrow \approx 80\%$) was observed in very large models (e.g., PaLM 540B in the original study). Smaller models did not improve as dramatically.

Chain-of-Thought (CoT) Prompting

- There are two main strategies for using Chain-of-Thought.
- Manual CoT: The prompt engineer manually write example questions along with step-by-step reasoning demonstrations that show how to solve them. These act as few-shot examples that teach the model how to reason for similar problems.
- Auto-CoT: Rather than the prompt engineer writing reasoning examples, the model automatically generates reasoning demonstrations for a set of representative questions. This is done by first clustering similar questions, then having the model create reasoning chains for one example from each cluster. These auto-generated examples are then used as prompts for the target task.

Advanced Reasoning and Reliability- Engineering Reasoning

- Manual CoT (User-provided structure)
 - Prompt includes an explicit template:
 - Step 1: Identify known quantities
 - Step 2: Identify assumptions
 - Step 3: Apply logic
 - Step 4: Conclude
- Auto-CoT (Model-generated reasoning)
 - Prompt: "Let's think step by step."
 - The model decides:
 - How many steps
 - What structure to follow

Hallucination and Grounding

- When we move from demos to real-world systems, the biggest challenge is not fluency, but it is reliability.
- Language models are excellent at producing plausible text. But plausible is not equal to true text.
- Hallucination is when a model produces incorrect or fabricated information confidently.
- Grounding means forcing the model to anchor its answer to verifiable information.

Hallucination

- Two major types:
 - A. Fact Fabrication (Invented Data)
 - This happens when the model:
 - Makes up statistics
 - Invents citations
 - Creates fake research papers
 - Fabricates historical events
 - Example: "A 2023 Harvard study proved that drinking coffee doubles memory retention."
 - Sounds believable. May be completely false.
 - The model is filling gaps with patterns.

Hallucination

- Two major types:
 - B. Logic Errors (Wrong Reasoning)
 - Here the facts may be correct but the reasoning is flawed.
 - Example: 5 shirts dry in 4 hours → 10 shirts dry in 8 hours.
 - No invented data, just incorrect reasoning.

Type	What Fails
Fact Fabrication	Knowledge accuracy
Logic Error	Reasoning process

Grounding

- Grounding means forcing the model to anchor its answer to verifiable information.
 - A. Source Attribution
 - B. RAG (Retrieval-Augmented Generation)
 - C. Chain of Verification (CoVe)

Grounding: Source Attribution

- Prompt technique: “Answer only using the provided text.”
 - This prevents the model from using external assumptions.
 - Why it works:
 - Constrains the context window
 - Reduces imagination
 - Encourages evidence-based answers
- Best for:
 - Academic settings
 - Legal review
 - Compliance checks

Grounding: RAG

- RAG stands for Retrieval-Augmented Generation
 - Instead of relying only on what the model “remembers,” these:
 - Retrieve relevant documents from a database
 - Inject them into the prompt
 - Ask the model to answer using those documents
 - So the model now has a reference book open in front of it.
 - Why This Matters
 - Without RAG: Model guesses from training memory
 - With RAG: Model reads specific, up-to-date information and answers are grounded in actual documents
 - This is widely used in:
 - Enterprise chatbots
 - Knowledge assistants
 - Medical or legal AI systems

Grounding: CoVe

- CoVe is also known as Chain of Verification
 - Instead of trusting the first answer, these:
 1. Ask the model to generate verification questions about its own claims
 2. Make it answer those questions
 3. Revise the final response
 - It's like asking:
 - "How do you know that?"
 - "Can you check that fact?"
 - This reduces hallucinated facts because the model is forced to re-examine assumptions.

The “I Don’t Know” Token

- One of the most important safety features in enterprise AI systems.
- Instead of forcing an answer, we train or prompt the model to:
 - “If you are unsure, say: I don’t know.”
- This prevents confident misinformation, builds user trust, and is essential in medical, legal, and financial domains
- In reliability engineering, a safe refusal is better than a confident lie.

The “I Don’t Know” Token

Problem

Fabricated facts:

Outdated information:

Overconfident answers:

Subtle inaccuracies:

Solution

Source restriction + RAG

RAG

“I don’t know” prompting

Chain of Verification

The Taxonomy of Intent: Open vs. Closed Prompting

- Effective prompt design requires a systematic understanding of the "Intent Taxonomy."
- This classifies prompts based on the degree of constraint placed on the model's output.
- Drawing from psychological and educational theories such as Bloom's Taxonomy and Scott's (1973) framework, prompt engineers categorize inputs primarily as either "Open" or "Closed".

Open-Ended Prompts: Generative Exploration

- Open-ended prompts are designed to elicit a wide range of possible responses.
- They are characterized by their lack of fixed answers, encouraging the model to engage in critical thinking, brainstorming, or creative synthesis.
- Objective: To generate detailed, creative, or critical thinking responses.
- Mechanism: These prompts set a broad topic or theme and allow the model's internal probability distribution to traverse a large semantic space.
- Applications: Ideal for brainstorming, exploratory essays, and complex problem solving where no single "correct" answer exists.
- Examples:
 - "What are the advantages and disadvantages of social media?"
 - "Explain how social engineering attacks can be identified."
 - "How can GenAI tools support education?"
 - "Write a gothic-punk style piece about an immortal being watching their city evolve over centuries."

Closed-Ended Prompts: Convergence and Precision

- Closed-ended prompts are direct queries that restrict the answer to a single correct response, a limited set of facts, or a specific categorical choice.
- Objective: To obtain high-fidelity, factual, or binary (Yes/No) data points.
- Mechanism: These prompts impose rigid boundaries on the model, forcing it to focus its attention on a narrow factual subset of its training data.
- Applications: Ideal for data extraction, verification, multiple-choice testing, and structured information retrieval.
- Examples:
 - "Is exercise important for a healthy lifestyle? (Yes/No)"
 - "The process of converting plain text to unreadable code is called..."
 - "Choose one: which protocol is used for secure web communications? (HTTP / HTTPS / FTP)"
 - "Identify five key components of a computer's central processing unit (CPU)."

Open Vs Closed-Ended Prompts

Feature	Open-Ended Prompts	Closed-Ended Prompts
Output Variance	High (Diverse responses)	Low (Convergent responses)
Cognitive Load	High (Requires synthesis/creation)	Low (Requires recall/matching)
Common Verbs	"Explain," "Discuss," "Create," "Hypothesize"	"Identify," "List," "Is," "Name"
Risk	Higher risk of irrelevance or fluff	Higher risk of technical omission if context is sparse